

Konstruksi Perangkat Lunak

PERTEMUAN 5

Design by Contract, Assertion, and Defensive Programming



Ekspektasi Luaran

Mampu menerapkan aspek reliabilitas pada source code perangkat lunak.



Outline

- Motivasi (contoh masalah)
 - Assertion
 - Design by Contract
 - Defensive Programming
- 

Design by Contract, Assertion, and Defensive Programming

Preface: Motivasi

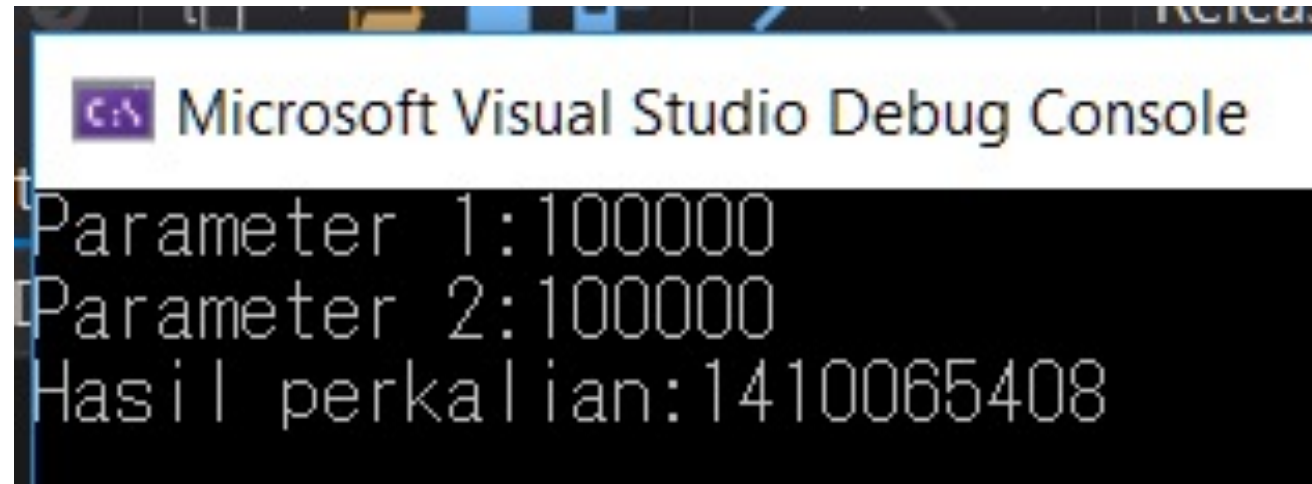


Berapakah nilai hasil perkalian di bawah ini?

```
static void Main(string[] args)
{
    int param1 = 100000;
    int param2 = 100000;

    Console.WriteLine("Parameter 1:" + param1);
    Console.WriteLine("Parameter 2:" + param2);
    Console.WriteLine("Hasil perkalian:" + param1 * param2);
}
```

Hasilnya bukan 10 Miliar!



```
Microsoft Visual Studio Debug Console  
Parameter 1:100000  
Parameter 2:100000  
Hasil perkalian:1410065408
```

Mengapa?



Mengapa?

Range nilai yang dapat diterima oleh tipe data
int pada C# adalah:

-2.147.483.648 sampai 2.147.483.647

Operasi matematis yang di luar range dapat
menyebabkan kesalahan perhitungan!





Dan ini menjadi masalah besar!

On December 25, 2004, Comair airlines was forced to ground 1,100 flights after its flight crew scheduling software crashed. The software used a 16-bit integer (max 32,768) to store the number of crew changes. That number was exceeded due to bad weather that month which led to numerous crew reassignments.

Masalah-Masalah Reliabilitas

- Range overflow: nilai yang dimasukkan pada sebuah variable melebihi range tipe data.
- Floating point: nilai pembagian pada float/double selalu mengalami pembulatan jika tidak berujung.
- Null pointer: penggunaan object reference yang tidak terinisialisasi.
- String index out of range: panjang string yang melebihi batas maksimal string.

Dan lebih banyak masalah baik dari batasan teknis maupun dari segi pengguna!



Design by Contract, Assertion, and Defensive Programming

Defensive Programming



Defensive Programming

“In defensive programming, **the main idea is that if a routine is passed bad data, it won't be hurt, even if the bad data is another routine's fault.** More generally, it's the recognition that programs will have problems and modifications, and that a smart programmer will develop code accordingly.”



Mengamankan Program dari Input yang Tidak Valid

Ada tiga cara umum untuk memastikan input tidak valid tidak masuk:

- Memastikan nilai dari seluruh data yang berasal dari sumber luar. (File, SQL command, dll.)
 - Memastikan nilai dari seluruh data yang berasal dari proses lain. (method, fungsi, dan prosedur lainnya)
 - Tentukan bagaimana menangani input yang buruk.
- 

Teknik-teknik Error Handling

- Isi return dengan nilai yang netral (0, "", NULL).
 - Untuk record, ganti dengan data berikutnya yang valid.
 - Isi return nilai yang sama dengan yang sebelumnya.
 - Ganti dengan nilai legal yang terdekat.
 - Log warning message ke dalam sebuah file.
 - Isi return dengan error code.
 - Panggil object/prosedur yang berfungsi khusus untuk melakukan error-processing.
 - Tampilkan error message
 - Matikan program.
- 

Design by Contract, Assertion, and Defensive Programming

Assertion



Assertion

“An assertion is **code that’s used during development**—usually a routine or macro—**that allows a program to check itself as it runs.** When an assertion is true, that means everything is operating as expected. When it’s false, that means it has detected an unexpected error in the code.”



Contoh Assertion

```
class Penduduk
{
    String NIK { get; }
    String Nama { get; }

    public Penduduk(String NIK, String nama)
    {
        //Penggunaan assertion untuk memastikan input sesuai
        Debug.Assert(NIK.Length == 16, "Panjang NIK tidak sesuai");
        Debug.Assert(nama.Length < 16, "Nama terlalu panjang");

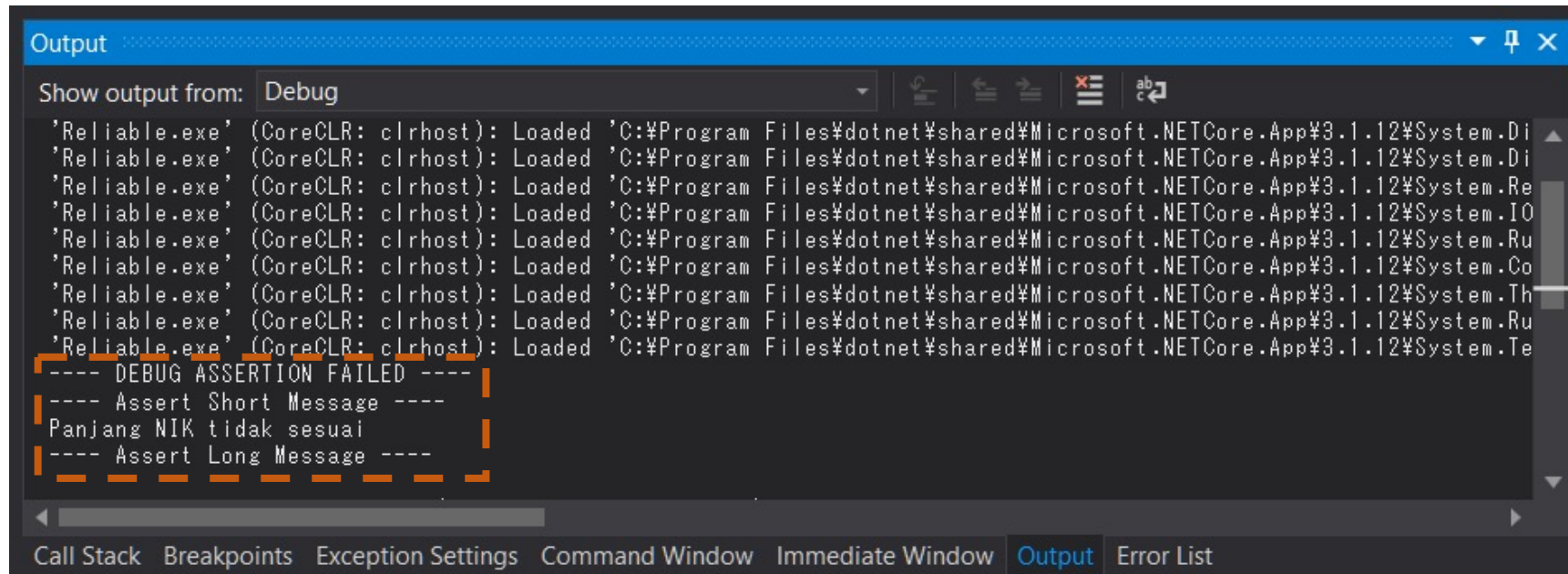
        this.NIK = NIK;
        Nama = nama;
    }
}
```


Contoh Failed Assertion

```
1 reference
14 public Penduduk(String NIK, String nama)
15 {
16     //Penggunaan assert untuk memastikan input sesuai
17     Debug.Assert(NIK.Length == 16, "Panjang NIK tidak sesuai");
18     Debug.Assert(nama.Length < 16, "Nama terlalu panjang");
19
20     this.NIK = NIK;
21     Nama = nama;
```

Eksekusi program berhenti di baris kode assert yang failed.

Contoh Failed Assertion



```
Output
Show output from: Debug
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Di
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Di
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Re
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.IO
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Ru
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Co
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Th
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Ru
'Reliable.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.12\System.Te
---- DEBUG ASSERTION FAILED ----
---- Assert Short Message ----
Panjang NIK tidak sesuai
---- Assert Long Message ----
```

Message assertion muncul pada window Output di IDE

Kapan Assertion Digunakan

- Gunakan error-handling untuk kondisi yang diperkirakan akan terjadi, gunakan assertion untuk kondisi yang tidak pernah terjadi.
 - Hindari memasukkan *executable code* kedalam blok assertion.
 - Gunakan assertions untuk mendokumentasikan dan memverifikasi *preconditions* dan *postconditions*.
 - Untuk code dengan kebutuhan keandalan yang sangat tinggi, gunakan assert dan error-handling secara bersamaan.
- 

Design by Contract, Assertion, and Defensive Programming

Design by Contract



Konsep contract

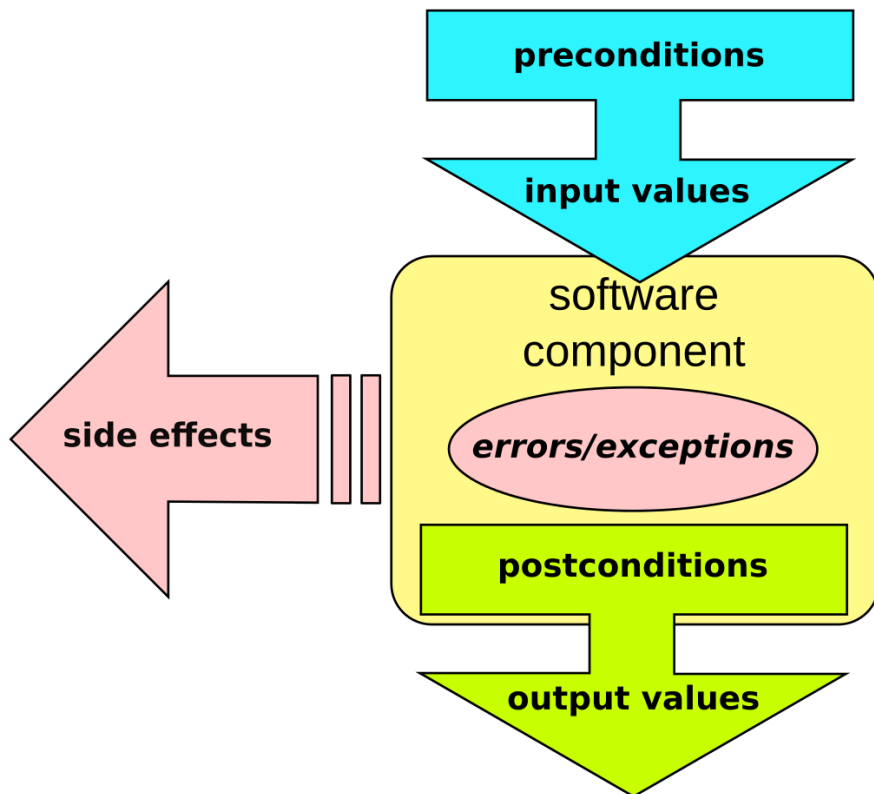
“Input yang akan saya berikan bertipe ... dengan nilai yang valid dalam range ... hingga ...”



“Output yang akan saya berikan bertipe ... dengan nilai berupa hasil operasi ... “

“Jika contract yang kami buat tidak kami penuhi, maka operasi tidak akan berjalan.”

Skema Design by Contract (DbC)



Apakah anda masih ingat konsep initial state dan final state pada mata kuliah dasar pemrograman?

Contoh Design by Contract (DbC)

```
public static int Multiply(int param1, int param2)
{
    //precondition
    Debug.Assert(param1 <= int.MaxValue && param1 >= int.MinValue);
    Debug.Assert(param2 <= int.MaxValue && param2 >= int.MinValue);

    //exception
    int result = checked(param1 * param2);

    //postcondition
    return result;
}
```

Q&A

Any questions or comments?



Further reading

- SWEBOK V3.0
 - S. McConnell, Code Complete 2nd Edition, Microsoft Press, 1993. Chapter 8
 - Bertrand Meyer, [Applying "Design by Contract"](#), IEEE Computer, October 1992.
- 